

PROJECT 3(SOLUTION)

STATEMENT: Design a 4 bit wide single – port RAM with Synchronous read/write and asynchronous reset.

The objective of this project is to design a **4-bit wide Single Port RAM** with 4 memory locations using Verilog.

A Single Port RAM is a memory block that uses a **single shared port** for both #read and #write operations — meaning only one operation can happen at a time.

Specifications from the problem statement:

- Data width: 4 bits
- Memory depth: 4 locations
- Address width: 2 bits ($2^2 = 4$ locations)
- Single port for both read and write
- Write: synchronous, occurs on posedge clk when we=1
- Read: synchronous, occurs on posedge clk when we=0
- Reset: asynchronous, clears all memory to 0 immediately when rst=1

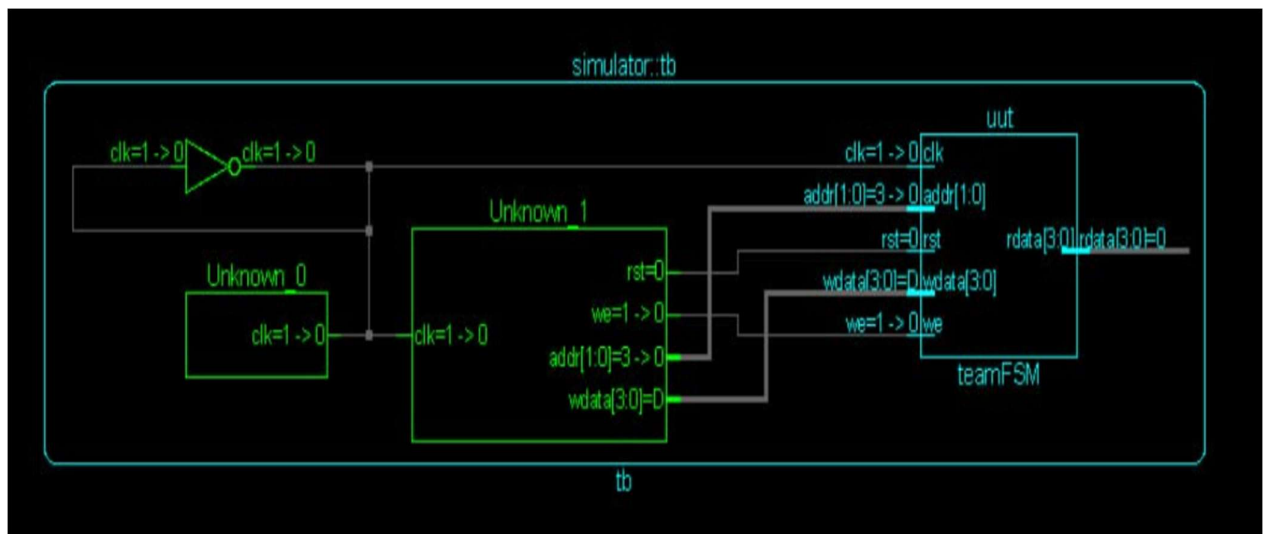
Port Table:

Port	Direction	Width	Function
clk	Input	1 bit	Clock signal
rst	Input	1 bit	Active-high async reset
we	Input	1 bit	Write enable
addr	Input	2 bits	Selects memory location
wdata	Input	4 bits	Data to write
rdata	Output	4 bits	Data read out

LOGIC USED:

1. If rst=1 then reset everything immediately (async).
2. Else if we=1 → write wdata into the memory address (sync).
3. Else read the data from memory address into rdata (sync)

BLOCK DIAGRAM:



HDL(VERLILOG CODE):

```
module teamFSM (  
    input clk,  
    input rst,  
    input we,  
    input [1:0] addr,  
    input [3:0] wdata,  
    output reg [3:0] rdata
```

```

);

reg [3:0] R_mem [0:3];

integer i;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        for (i = 0; i < 4; i = i + 1)begin
            R_mem[i] <= 4'b0000;
        end
        rdata <= 4'b0000;
    end
    else if (we)begin
        R_mem[addr] <= wdata;
    end
    else begin
        rdata <= R_mem[addr];
    end
end
endmodule

```

CODE EXPLATION:

#1.

1. module teamFSM (
2. input clk, rst, we,
3. input [1:0] addr,
4. input [3:0] wdata,

5. output reg [3:0] rdata
6.);

- Declares all input/output ports
- rdata is reg because it is assigned inside always block

#2.

1. reg [3:0] R_mem [0:3];
2. integer i;

- R_mem is the actual memory array — 4 locations, each 4 bits
- integer i is a loop counter used only during reset

#3.

1. **always** @(posedge clk or posedge rst) begin
- Triggers on two events: rising clock edge OR rising rst
 - rst in sensitivity list makes reset **asynchronous**

#4.

1. if (rst) begin
 2. for (i = 0; i < 4; i = i + 1) begin
 3. R_mem[i] <= 4'b0000;
 4. end
 5. rdata <= 4'b0000;
 6. end
- When rst=1, loop clears all 4 memory locations to 0
 - rdata also cleared to avoid stale output
 - Loop unrolls to 4 parallel assignments in hardware

#5.

1. else if (we) begin
 2. R_mem[addr] <= wdata;
 3. end
- When rst=0 and we=1 → write wdata into selected location
 - Happens on posedge clk (synchronous)

#6.

else begin

 rdata <= R_mem[addr];

end

- When rst=0 and we=0 → read selected location into rdata
- Also synchronous — valid data appears one cycle after address

TESTBENCH DETAILS:

`timescale 1ns/1ps

module tb;

 reg clk, rst, we;

 reg [1:0] addr;

 reg [3:0] wdata;

 wire [3:0] rdata;

 teamFSM uut (

```
.clk(clk),  
.rst(rst),  
.we(we),  
.addr(addr),  
.wdata(wdata),  
.rdata(rdata)  
);
```

```
initial clk = 0;  
always #5 clk = ~clk;
```

```
initial begin
```

```
    $monitor("Time=%0t | rst=%b | we=%b | addr=%b | wdata=%h |  
rdata=%h", $time, rst, we, addr, wdata, rdata);
```

```
    rst = 1; we = 0; addr = 0; wdata = 0;  
    #7 rst = 0;
```

```
//data writing
```

```
    @(negedge clk); we = 1; addr = 2'b00; wdata = 4'b1010;  
    @(posedge clk); #1;
```

```
    @(negedge clk); we = 1; addr = 2'b01; wdata = 4'b1011;  
    @(posedge clk); #1;
```

```
@(negedge clk); we = 1; addr = 2'b10; wdata = 4'b1100;
```

```
@(posedge clk); #1;
```

```
@(negedge clk); we = 1; addr = 2'b11; wdata = 4'b1101;
```

```
@(posedge clk); #1;
```

```
//data reading
```

```
@(negedge clk); we = 0; addr = 2'b00;
```

```
@(posedge clk); #1;
```

```
@(negedge clk); we = 0; addr = 2'b01;
```

```
@(posedge clk); #1;
```

```
@(negedge clk); we = 0; addr = 2'b10;
```

```
@(posedge clk); #1;
```

```
@(negedge clk); we = 0; addr = 2'b11;
```

```
@(posedge clk); #1;
```

```
//checking reset
```

```
rst = 1;
```

```
#7 rst = 0;
```

```
@(negedge clk); we = 0; addr = 2'b00;
```

```
@(posedge clk); #1;
```

```
$finish;  
  
end  
  
endmodule
```

WAVEFORM ANALYSIS:

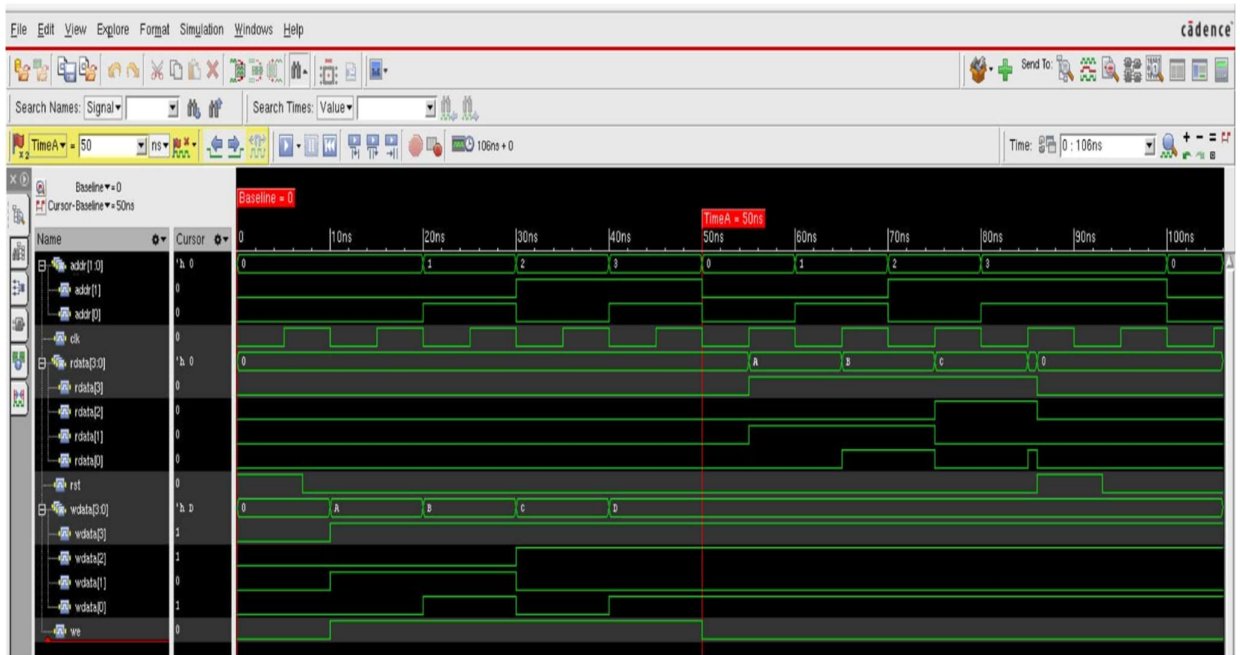
1.0–7ns: rst=1 is applied. All memory and rdata cleared to 0. This confirms asynchronous reset is working correctly.

2.7–50ns (Write phase): we=1 is active. addr increments from 00→01→10→11 each clock cycle. wdata values are written sequentially. rdata stays 0 during write phase since we=1.

3.50–90ns (Read phase): we=0. addr steps through 00→01→10→11. rdata outputs one cycle after each address is applied. This confirms synchronous read with 1-cycle latency.

4. rst=1 applied again. rdata immediately drops to 0 — confirming asynchronous reset clears output without waiting for clock edge.

5.Observation: The 1-cycle reading is clearly visible in the waveform — rdata updates one clock edge after addr and we=0 are applied.



#

Stimulus sequence:

Step	Operation	addr	wdata/rdata
1	Reset	—	All cleared to 0
2	Write	00	1010
3	Write	01	1011
4	Write	10	1100
5	Write	11	1101
6	Read	00	1010
7	Read	01	1011
8	Read	10	1100
9	Read	11	1101
10	Reset again	—	All cleared to 0
11	Read after reset	00	0000

CHALLENGES FASED:

Challenge 1: Understanding why addr is input not output

While writing the port list, there was confusion about whether addr should be input or output. Since the testbench drives the address to tell the RAM where to read or write, addr must always be an input. The RAM itself never generates its own address.

Challenge 2: Understanding the for loop during reset

Since R_mem has 4 separate locations, all 4 must be individually cleared. Writing R_mem <= 0 alone does not work on an array. The for loop was needed to iterate through each location. Understanding that this loop gets unrolled into 4 parallel hardware assignments during synthesis was an important concept to grasp.